

Lecture Slides

Object Oriented Programming

**Assignment Operator, Copy Constructor,
Avoid Code Duplication**

AGENDA

The agenda of today's lecture is:

1. Assignment Operator
2. Copy Constructor
3. Avoid Code Duplication
4. Practice

1. Assignment Operator

The assignment operator (operator=) is used to copy the contents of one object into another existing object of the same class.

```
ClassName& operator=(const ClassName& other);
```

Assignment Operator Example

```
class Point {  
    int x, y;  
public:  
    Point(int a = 0, int b = 0){  
        x = a, y = b;  
    }  
    void show() const {  
        cout << "(" << x << ", "  
            << y << ")" << endl;  
    }  
};
```

```
int main() {  
    Point p1(2, 3);  
    Point p2(5, 6);  
    p1.show();  
    p2.show();  
    p2 = p1;  
    p2.show();  
    return 0;  
}
```

2. Shallow Copy vs Deep Copy

A default copy mechanism is available in all classes

By default, this mechanism performs a shallow copy.

This works fine if the object only contains members stored on the stack.

Issues arise when the object contains raw pointers to heap memory.

In this scenario, a shallow copy copies only the pointer's address, resulting in both objects sharing the same memory block on the heap.

Consequently, any modification to the heap data made via one object will be immediately visible in the other.

Shallow Copy Example

```
class ShallowIssue {  
    int* data;  
public:
```

```
int main() {  
    ShallowIssue a(10);  
    ShallowIssue b = a;  
    b.set(20);  
    a.show();  
}
```

```
    ShallowIssue(int val) { data = new int(val); }  
    void set(int d) { *data = d; }  
    ~ShallowIssue() { delete data; }  
    void show() const { cout << *data << endl; }  
};
```

Deep Copy

A deep copy assigns a separate copy of memory on heap

Copy elements on heap from one object to other

Each object has separate copy of the object

No data security risk

No duplicate memory deletion

Deep Copy Example

```
class Deep {  
    int* data;  
public:  
    Deep(int val) { data = new int(val); }  
    ~Deep() { delete data; }  
    Deep(const Deep& other) {  
        data = new int;  
        *data = *other.data;  
    }  
};
```


3. Assignment Operator and Copy Constructor

Why both required:

- * copy constructor called when new object created
- * assignment operator called when copy is made in existing object

Copy Constructor and Assignment Operator Example

```
class CC {  
    int* ptr;    int size;  
public:  
    CC(int s) {  
        size = s;  
        ptr = new int[size];  
    }  
    CC(const CC& other) {  
        size = other.size;  
        ptr = new int[size];  
        for (int i = 0 ; i < size ; i++)  
            ptr[i] = other.ptr[i];  
    }  
};
```

```
CC& operator=(CC& o) {  
    delete []ptr;  
    size = o.size;  
    ptr = new int[size];  
    for (i=0 ;i<size ; i++)  
        ptr[i] = o.ptr[i];  
}
```

4. Use of Private Utility Function to Avoid Code Duplication

When both the copy constructor and assignment operator perform similar operations, repeating the same code can lead to maintenance issues and potential mistakes.

To avoid redundancy and make the code more readable, private utility functions like `copy ()` is used. This ensures that copy logic are written only once, keeping the class concise and easier to modify.

Code Duplication Avoidance Example

```
class S{
    int* ptr;
    S& copy (const S& o){
        size = other.size;
        ptr = new int[size];
        for (i=0;i<size;i++)
            ptr[i]=o.ptr[i];
        return *this;
    }
public:
```

```
S (const S& other) {
    copy (other);
}
S& operator=(S& other){
    delete []ptr;
    return copy(other);
}
```

4. Use of Private Utility Function to Avoid Code Duplication

When both the copy constructor and assignment operator perform similar operations, repeating the same code can lead to maintenance issues and potential mistakes.

To avoid redundancy and make the code more readable, private utility functions like `copy ()` is used. This ensures that copy logic are written only once, keeping the class concise and easier to modify.

5. Use of Private Utility Function to Delete Memory

The purpose is to avoid repeating deletion logic in destructor and assignment operator.

This is extremely useful, when multiple pointer members pointing to heap memory or a double pointer pointing to complex dynamic memory, thus it helps to write complex code once only.

Code Duplication Avoidance in Delete Example

```
void cleanUp(){
    delete data; data=nullptr;
}
public:
    ~Sample() {
        cleanUp();
    }
    Sample& operator=(const Sample& other) {
        cleanUp();
        return copyFrom(other);
    }
}
```

6. Comprehensive Example

The **MyArray** class is designed to handle arrays of dynamic and unlimited size

The class contains three data members: *size*, *current size* and a pointer member that points to a dynamically allocated array

Inserting a new element check, whether there is available space. Yes, the element is added, and *currentSize* is incremented.

If the array is full, the class dynamically allocates a new larger array. It copies all elements to new array.

The old array is deleted to free memory.

The pointer is updated to reference the new array, and size is updated accordingly.

The new element is then added, and currentSize is incremented.

7. Practice Tasks

7.1. Player Class: Implement a class that stores a player's performance across multiple matches. The class should include data members for the number of matches and two integer pointers — one to dynamically store runs scored in each match and the other to store balls played in each match. Properly implement the copy constructor, assignment operator, and destructor to ensure deep copying of all data so that each player object maintains its own independent record of runs and balls without sharing memory with others.

7.2. Book Class: Create a class that represents a book with dynamically allocated memory for the title (character pointer) and author (character pointer). Implement a copy constructor, assignment operator, and destructor to handle deep copying and prevent memory leaks.

7. Practice Tasks (cont...)

7.3. Student Class:

Implement a class that stores a student's information, including the student ID, number of subjects, and a dynamically allocated array of grades (integer pointer). Ensure proper deep copy in the copy constructor and assignment operator so that grades of one student are not shared with another.

7.4. Employee Class: Design a class with a name (character pointer), monthly salary records (integer pointer), and number of months. Each employee object should maintain its own salary data through deep copying. Include destructor cleanup to avoid dangling pointers.